KUBERNETES PREPARATION · MY LEARNING NOTES · SESSION 09

Storage → Persistent Storage (PV, PVC, StorageClass)

Your notes, cleaned up from dictation + whiteboard + screenshots

Quick recall

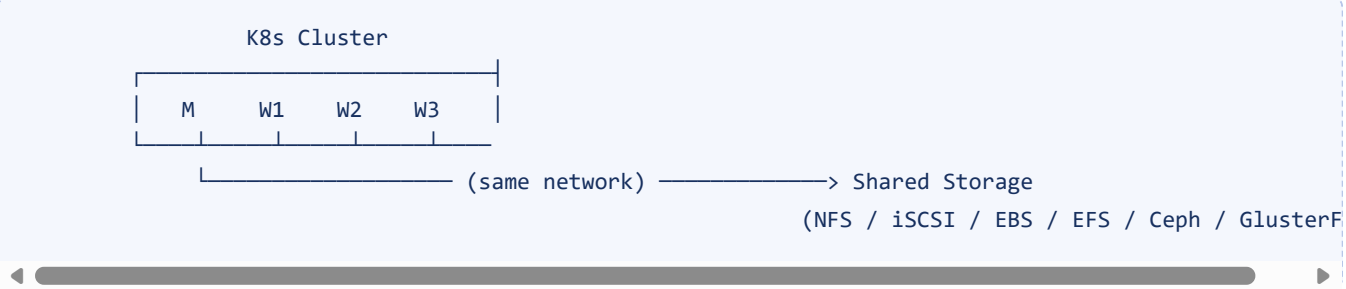
- A Docker volume only solves storage for **one container on one machine**. Kubernetes needs storage that **any pod, on any worker node, can reach** — that's what Persistent Volumes exist for.
- **Persistent Volume Subsystem** = 3 objects: **PV** (Persistent Volume), **PVC** (Persistent Volume Claim), **SC** (Storage Class).
- A Pod never talks to a PV directly. The chain is always: `Pod → PVC → PV → external storage`. Think of a **PVC as a ticket to use a PV**.
- **Static provisioning** = admin manually creates the PV first. **Dynamic provisioning** = a StorageClass auto-creates the PV the moment a PVC asks for one — no manual PV step at all.

PART 1 · WHY PERSISTENT VOLUMES EXIST

The Docker-volume limitation

In plain Docker, a container writes to a mounted volume on the **same host**. That's fine until the container is scheduled somewhere else — a different machine simply can't see that volume. In a Kubernetes cluster (1 manager + workers W1/W2/W3), if a pod writes to storage created **only on W1**, W2 and W3 have no access to it at all.

The fix: a **shared storage** that lives outside the Kubernetes cluster, but on the *same network* as the cluster, reachable by every worker node via its IP — regardless of which node a pod actually lands on.



CSI — Container Storage Interface

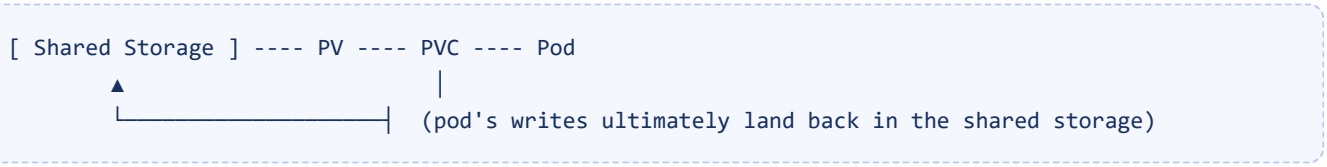
Kubernetes attaches this external storage through a **plugin-based architecture** called **CSI (Container Storage Interface)**. The storage itself can be anything: **NFS, iSCSI, EBS, EFS, Ceph, GlusterFS** — CSI is the common contract that lets Kubernetes talk to any of them the same way.

PART 2 · THE PERSISTENT VOLUME SUBSYSTEM

PV, PVC, SC

```
Persistent Volume Subsystem
 ├─ PV  – Persistent Volume
 ├─ PVC – Persistent Volume Claim
 └─ SC  – Storage Class
```

- **PV (Persistent Volume):** once external shared storage becomes *visible inside* the Kubernetes cluster, that visible representation is called a PV.
- **PVC (Persistent Volume Claim):** a Pod **cannot** access a PV directly. A PVC sits between them — think of a **PVC as a ticket to use a PV**.
- **SC (Storage Class):** covered in Part 4 — the mechanism that makes PV creation automatic.



Size — the block-device rule

- **One block device = one PV.** You cannot carve a single 100Gi block device into two separate PVs (e.g. 50Gi + 50Gi) — that's not how it works.
- A PV can be created *smaller* than the underlying block device (e.g. an 80Gi PV out of a 100Gi device), but the remaining 20Gi is simply wasted — unusable by anything else.
- A PVC can request **less than or equal to** its PV's size (e.g. a 70Gi PVC against a 100Gi PV) — again, the leftover capacity is wasted, not reusable by another PVC.

Permissions — 3 access modes

Mode	Shorthand	Meaning
Read Write Many	**RWX**	Many nodes/pods can read *and* write at once
Read Only Many	**ROX**	Many nodes/pods can read; none can write

Read Write Once	**RWO**	Only one node can read/write at a time

> ⚠ **Golden rule:** *a PV can only be accessed through one permission mode at a time* — and that mode is dictated by what the **underlying storage technology actually supports**, not by preference. A PVC's access mode must match the PV it binds to.

- Storage that supports RWX (e.g. **NFS**) → its PV can be configured RWX → PVC can also be RWX.
- Storage that only supports RWO (e.g. **EBS**) → its PV can only be RWO → PVC must also be RWO.
- You cannot request an access mode from a PV that its backing storage doesn't actually support.

Which mode do you actually need? It's driven entirely by the application:

- Many pods need to write concurrently (e.g. a shared upload directory) → need **RWX**.
- Pods only ever read, never write → **ROX** is enough.
- Only a single pod/node ever touches this volume → **RWO** is fine.

Volume plugin support matrix (from the Kubernetes docs, as referenced this session):

Volume Plugin	RWO	ROX	RWX	RWOP
AzureFile	✓	✓	✓	–
CephFS	✓	✓	✓	–
CSI	depends on driver	depends on driver	depends on driver	depends on driver
FC	✓	✓	–	–
FlexVolume	✓	✓	depends on driver	–
GCEPersistentDisk	✓	✓	–	–
Glusterfs	✓	✓	✓	–
HostPath	✓	–	–	–
iSCSI	✓	✓	–	–
NFS	✓	✓	✓	–
RBD	✓	✓	–	–
VsphereVolume	✓	–	works only if Pods are collocated	–
PortworxVolume	✓	–	✓	–

This is exactly why **NFS was used for this session's examples** — it's one of the few plugins that natively supports RWX.

Data retention policy — Retain vs. Recycle

Once an app stops using a PVC (deleted, decommissioned), what happens to the data already written?

- **Retain** — keep the data. Safer default for anything you can't afford to lose.
- **Recycle** — the data gets cleared/reused.

(This maps to the more current `Delete` / `Retain` reclaim policy values covered in the master notes — `Recycle` is the older, now-deprecated value, but the underlying decision — keep data vs. let it go — is the same idea.)

PART 3 · STATIC PROVISIONING, WORKED END TO END (NFS)

Setup: 1 manager (`10.0.0.100`) + 2 workers (`10.0.0.1` , `10.0.0.2`), plus a separate NFS server at `10.0.0.99` .

Step 1 — Prepare the NFS export (on the NFS server)

```
apt install nfs-kernel-server
mkdir --mode=777 /weekend
vim /etc/exports
# /weekend *(rw,sync,no_subtree_check)

exportfs -r
exportfs -a
showmount -e localhost
# Export list for localhost:
# /weekend *

systemctl status nfs-kernel-server
```

Step 2 — Confirm the cluster can see the export (on the manager)

```
showmount -e 10.0.0.99
# -bash: /usr/sbin/showmount: No such file or directory   ← nfs-common not installed ye

apt install nfs-common

showmount -e 10.0.0.99
# Export list for 10.0.0.99:
# /weekend *
```

Confirmed on a worker node too (`showmount -e 10.0.0.99` from `nodetwo`) — the export is visible cluster-wide, exactly as expected for shared storage on the same network.

Step 3 — Create the PV manually

```
kubectl get pv
# No resources found
```

```
# nfspv.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pvone-nfs
spec:
  capacity:
    storage: 5Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Recycle
  storageClassName: slow
  nfs:
    path: /weekend
    server: 10.0.0.99
```

```
kubectl create -f nfspv.yaml
# persistentvolume/pvone-nfs created

kubectl get pv
# NAME        CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS     CLAIM  STORAGECLASS  AG
# pvone-nfs   5Gi       RWO           Recycle         Available         slow          3s

kubectl describe pv pvone-nfs
# Source:
#   Type:      NFS (an NFS mount that lasts the lifetime of a pod)
#   Server:    10.0.0.99
#   Path:      /weekend
#   ReadOnly:  false
# Claim:                          ← empty: nobody has claimed it yet
# Status:      Available
```

Step 4 — Create the PVC

```
# pvc-nfs.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  volumeMode: Filesystem
  resources:
    requests:
      storage: 4Gi
  storageClassName: slow
```

```
kubectl create -f pvc-nfs.yaml
# persistentvolumeclaim/myclaim created

kubectl get pv
# pvone-nfs   5Gi   RWO   Recycle   Bound   default/myclaim   slow   85s   ← flips from

kubectl get pvc
# NAME      STATUS   VOLUME      CAPACITY   ACCESS MODES   STORAGECLASS   AGE
# myclaim   Bound    pvone-nfs   5Gi        RWO            slow           18s
```

Note the PVC requested `4Gi`, but it bound to the `5Gi` PV — matching `storageClassName` + sufficient capacity + compatible access mode is what makes a bind succeed; sizes don't have to be identical, just "PV ≥ PVC request."

Step 5 — Create a Pod that uses the PVC

```
# pvc-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
    - name: myfrontend
      image: lovelearnlinux/webserver:v1
      volumeMounts:
        - mountPath: "/var/www/html"
          name: webroot
  volumes:
    - name: webroot
      persistentVolumeClaim:
        claimName: myclaim
```

The volume mount's `name` must match the `volumes[].name` (`webroot` in both places) — that's the link between "where the container sees the mount" and "which PVC actually backs it."

```
kubectl describe pvc myclaim
# Status:        Bound
# Volume:        pvone-nfs
# Annotations:   pv.kubernetes.io/bind-completed: yes
#                pv.kubernetes.io/bound-by-controller: yes
# Used By:       <none>          ← before the pod exists

kubectl create -f pvc-pod.yaml
# pod/mypod created

kubectl get pods
# mypod   1/1   Running   0   25s

kubectl describe pod mypod
# Mounts:
#   /var/www/html from webroot (rw)
```

Step 6 — Prove the data really lands on NFS

```
kubectl exec -it mypod -- /bin/bash
[root@mypod /]# df -h
# 10.0.0.99:/weekend   20G   15G   3.2G   83%   /var/www/html

[root@mypod /]# cd /var/www/html
[root@mypod html]# echo "this file should go to nfs" > alok.html
```

On the NFS server itself:

```
cd /weekend
ll
# -rw-r--r-- 1 nobody nogroup 27 Oct 7 11:57 alok.html
```

And from the manager, hitting the pod's own webserver:

```
curl http://172.16.3.184/alok.html
# this file should go to nfs
```

The file written inside the pod's mount is the exact same file sitting on the NFS server — confirming the whole `Pod → PVC → PV → NFS` chain end to end.

PART 4 · DYNAMIC PROVISIONING WITH A STORAGECLASS

The problem static provisioning doesn't scale

Three application teams — **Dev, Test, Prod** (also framed as **Blue, Orange, Pink** in the whiteboard example) — each need their own storage. With static provisioning, for **every single team**, the storage team has to: carve a new block device, mount it, add an `/etc/exports` entry, publish it on the network — and then someone still has to hand-write a PV + PVC per team. Multiply this by every new app, and the storage team drowns in repetitive manual work.

The fix: one big volume + a StorageClass

Instead of one export per team, the storage team creates **one large volume** (e.g. 100Ti), mounts it as `/k8sdata`, and exports the whole thing once:

```
/etc/exports:  /k8sdata *(rw,sync,no_subtree_check)
```

On top of that single export, a **StorageClass** object is created once, wired to that NFS path/server. From then on, **any team just writes a PVC** naming that StorageClass + the size they want — Kubernetes **automatically creates the PV for them**. No more per-team manual PV creation, no more repeated trips to the storage team.



This is the entire point of dynamic provisioning: a PVC alone is enough; the PV appears "by magic" (a provisioner controller watching for unbound PVCs against that StorageClass).

Installing an NFS provisioner (Helm)

Vanilla Kubernetes has no built-in NFS dynamic provisioner, so one is installed via Helm (`nfs-subdir-external-provisioner`):

```
# Install Helm
curl https://baltocdn.com/helm/signing.asc | sudo apt-key add -
sudo apt-get install apt-transport-https --yes
echo "deb https://baltocdn.com/helm/stable/debian/ all main" | sudo tee /etc/apt/sources
sudo apt-get update
sudo apt-get install helm

# Add the provisioner's chart repo
helm repo add nfs-subdir-external-provisioner https://kubernetes-sigs.github.io/nfs-subd
helm repo list

# Install it into its own namespace
kubectl create ns storagenfs
helm install nfs-subdir-external-provisioner \
  nfs-subdir-external-provisioner/nfs-subdir-external-provisioner \
  --set nfs.server=10.0.0.99 \
  --set nfs.path=/k8sdata \
  --set storageClass.onDelete=true -n storagenfs
```

```
kubectl get storageclass
# NAME         PROVISIONER                                RECLAIMPOLICY  VOLUMEBINDINGMO
# nfs-client   cluster.local/nfs-subdir-external-provisioner  Delete      Immediate

kubectl describe storageclass nfs-client
# Provisioner:   cluster.local/nfs-subdir-external-provisioner
# Parameters:    archiveOnDelete=true,onDelete=true
```

`archiveOnDelete=true` means a deleted PVC's data folder gets **renamed to** `archived-...` on the NFS server instead of being wiped — a safety net against accidental data loss even under a `Delete`-reclaim StorageClass.

An app team's experience — just write a PVC

```
# app1-pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: sample-nfs-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: nfs-client
  resources:
    requests:
      storage: 2Gi
```

```
kubectl create -f app1-pvc.yaml
# persistentvolumeclaim/sample-nfs-pvc created

kubectl get pvc
# sample-nfs-pvc    Bound    pvc-a3f0da32-35ab-495c-8a3c-780bf4cc919d   2Gi    RWO    nfs-cl

kubectl get pv
# pvc-a3f0da32- ...    2Gi    RWO    Delete    Bound    default/sample-nfs-pvc    nfs-client
```

No PV was ever written by hand — it was auto-generated (name, capacity, everything) the moment the PVC referenced `storageClassName: nfs-client`. On the NFS server itself:

```
cd /k8sdata
ll
# archived-default-sample-nfs-pvc-pvc-a1d8cb44- ...     ← a previous, now-deleted PVC's d
# default-sample-nfs-pvc-pvc-a3f0da32- ...               ← this PVC's live folder
```

The provisioner names each folder `<namespace>-<pvc-name>-<pv-name>` — this is also how multiple teams' data stays cleanly separated under one shared `/k8sdata` export.

Using it from a real Deployment

```yaml
# dep.yaml
apiVersion: v1
kind: Service
metadata:
  name: nfs-nginx-svc
spec:
  selector:
    app: sc-nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
---
apiVersion: apps/v1
kind: Deployment
metadata:
  labels: { app: sc-nginx }
  name: nfs-nginx
spec:
  replicas: 2
  selector:
    matchLabels: { app: sc-nginx }
  template:
    metadata:
      labels: { app: sc-nginx }
    spec:
      volumes:
        - name: nfs-test
          persistentVolumeClaim:
            claimName: sample-nfs-pvc
      containers:
        - image: nginx
          name: nginx
          volumeMounts:
            - name: nfs-test          # must match volumes[].name above
              mountPath: /usr/share/nginx/html
              # readOnly: true        # optional: enforce read-only on this mount
          ports:
            - containerPort: 80
```

```
kubectl create -f dep.yaml
# service/nfs-nginx-svc created
# deployment.apps/nfs-nginx created

kubectl get pods
# nfs-nginx-b899fbfd-ht7rj   1/1   Running   0   11s
# nfs-nginx-b899fbfd-zjk2b   1/1   Running   0   11s

kubectl describe pod nfs-nginx-b899fbfd-ht7rj
# Mounts:
#   /usr/share/nginx/html from nfs-test (rw)
# Volumes:
#   nfs-test:
#     Type:       PersistentVolumeClaim (a reference to a PersistentVolumeClaim in the s
#     ClaimName:  sample-nfs-pvc

kubectl exec -it nfs-nginx-b899fbfd-ht7rj -- /bin/bash
[root@nfs-nginx- ... /]# df -h
# 10.0.0.99:/k8sdata/default-sample-nfs-pvc-pvc-a3f0da32- ...   29G   9.1G   19G   34%
```

Both replicas mount the exact same NFS-backed path — this is the practical payoff of `ReadWriteMany`-capable storage (NFS) behind a shared PVC: multiple pods, one shared, dynamically provisioned volume, with zero manual PV work by either the app team or the storage team.

Practice Tasks

☐ Stand up an NFS export, then run `showmount -e <nfs-ip>` from both the manager and a worker node — confirm the export is visible cluster-wide before writing any Kubernetes YAML.

☐ Manually create a PV + PVC (static provisioning) and confirm `kubectl get pv` flips from `Available` to `Bound` only after the matching PVC exists — not before.

☐ Deliberately request a PVC size larger than any existing PV's capacity — confirm it stays `Pending` and diagnose why via `kubectl describe pvc`.

☐ Write a file inside a pod's PVC-backed mount, then check it appears on the NFS server directly (and vice versa) — prove the round trip for yourself rather than trusting the mechanism blindly.

☐ Install the `nfs-subdir-external-provisioner` via Helm, create a PVC against its StorageClass with **no PV written by hand**, and confirm a PV appears automatically — inspect its auto-generated name.

☐ Delete a dynamically-provisioned PVC with `archiveOnDelete=true` set, and confirm the NFS server renames (rather than deletes) the corresponding data folder to `archived- ...`.

☐ Scale a Deployment backed by one shared PVC to 2+ replicas and confirm via `df -h` inside each pod that they all mount the identical NFS path — the concrete proof of `ReadWriteMany` in action.

What I practiced

- Set up an NFS server and export, confirmed visibility from the Kubernetes manager and a worker node via `showmount`.
- Statically provisioned storage end to end: hand-wrote a PV (`nfspv.yaml`), a PVC (`pvc-nfs.yaml`), and a Pod (`pvc-pod.yaml`), watched the PV transition `Available → Bound` , and proved data written inside the pod's mount landed on the real NFS export (and was independently readable via `curl` against the pod's own IP).
- Installed `nfs-subdir-external-provisioner` via Helm into its own namespace, created a `StorageClass` (`nfs-client`) backed by a single shared `/k8sdata` export, and created a PVC against it with **no manual PV step** — confirmed the PV was auto-generated.
- Deployed a 2-replica `nginx` Deployment sharing one dynamically-provisioned PVC, and confirmed via `kubectl exec` + `df -h` that both replicas mounted the exact same NFS-backed path.

Merged into the master reference notes: k8s-master-notes.md §8 (Storage) — added a "Persistent Volume Subsystem — Deep Dive" addendum covering access-mode rules, the block-device sizing rule, and a full static + dynamic (Helm NFS provisioner) provisioning walkthrough.